Behavior Pattern Code Analysis

Repo: https://eae-git.eng.utah.edu/u1537288/playerstatepattern/-/tree/main

Analyze your usage of the behavior pattern.

Which one did you choose, or did you do a combo? Why?

I implemented a Type Object Pattern to make any enemy system that uses a single enemy base class and an EnemyType data asset class. I wanted to do this pattern because I am very much interested in Enemy AI behaviors and different techniques to implement them (not just behavior or state trees). The type Object also makes creating several enemy types very seamless and straightforward to wear; even small variations like movement or attack damage can easily be changed. Since the enemy stats were primarily data-driven, there was no need for subclasses of enemies. This made it really easy to change animation montages as well because I could just create a variable to assign different ones in the editor. Creating more intelligent AI would need more than just a Type Object Pattern, especially when using a Behavior Tree. Although they integrate well together, which was my plan for Project 5.

Analyze your usage of other patterns

What pattern(s) did you use the most? What pattern(s) did you choose not to use?

Outside of the Type Object, I tried implementing the sandclass sandbox for a weapon with different fire modes. Although it works in principle by showing debug messages when switching modes, I decided to abandon it and focus on one behavior pattern for time constraints. The state pattern is also a part of the project, which is untouched by Project 3. Currently, there is no direct integration with how the type object pattern works, as it was more to showcase enemy types through data-driven assets rather than combat with them.

```cpp
UENUM(BlueprintType)
enum class EEnemyAttackType : uint8
{
    Melee,
    Ranged,
    Charger
};

UCLASS(BlueprintType)
class DESIGNPATTERNS_API UEnemyType : public UDataAsset
{
    GENERATED_BODY()

public:

    UPROPERTY(EditAnywhere, Category = "Stats")
    float MaxHealth = 100.f;

    UPROPERTY(EditAnywhere, Category = "Stats")
    float MoveSpeed = 300.f;

    UPROPERTY(EditAnywhere, Category = "Stats")
    Changed in 2 Blueprints
    float Damage = 10.f;

    UPROPERTY(EditAnywhere, Category = "Stats")
    float AttackRange = 150.f;

    UPROPERTY(EditAnywhere, Category = "Combat")
    float AttackCooldown = 1.5f;

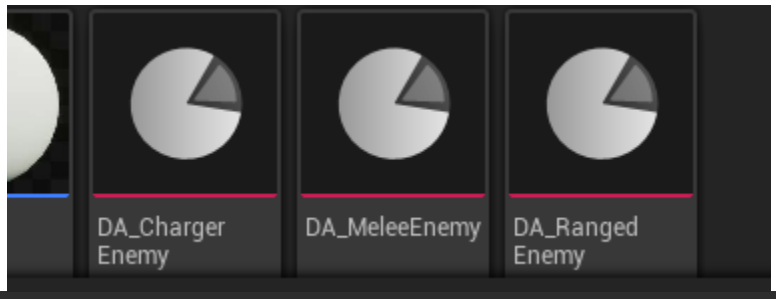
    UPROPERTY(EditAnywhere, Category = "Behavior")
    EEnemyAttackType AttackType;

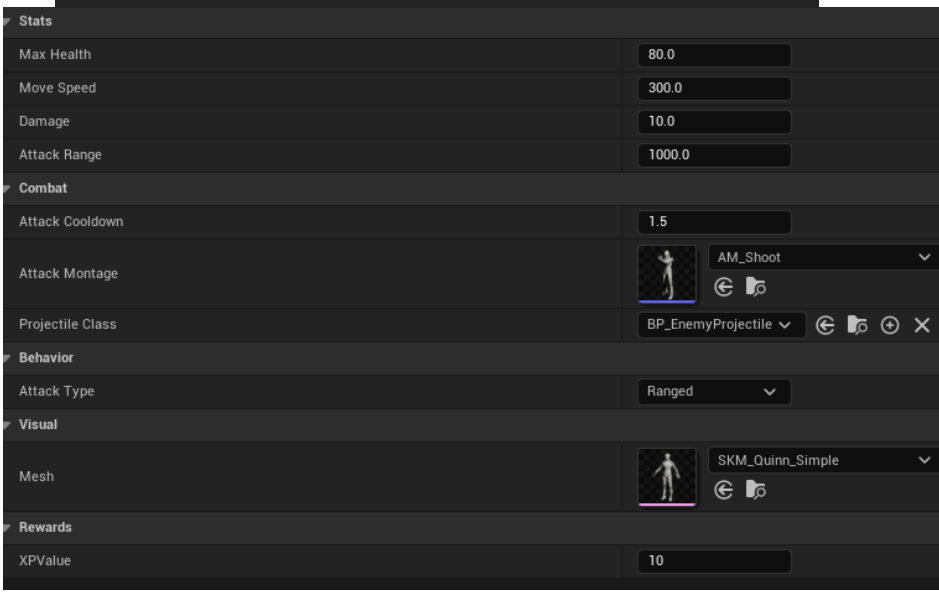
    UPROPERTY(EditAnywhere, Category = "Visual")
    Changed in 3 Blueprints
    USkeletalMesh* Mesh;

    UPROPERTY(EditAnywhere, Category = "Rewards")
    Changed in 0 Blueprints
    int XPValue = 10;

    UPROPERTY(EditAnywhere, Category = "Combat")
    Changed in 3 Blueprints
    UAnimMontage* AttackMontage;

    UPROPERTY(EditAnywhere, Category = "Combat")
    Changed in 1 Blueprint
    TSubclassOf<AActor> ProjectileClass;
};
```





Something important to note is that for combat-related games, although these patterns are great backbones to build off, the Unreal Gameplay Ability System would be better, and something I am currently exploring in my free time. There is a lot to learn, but it integrates well with animation times, as well as being able to select different weapon modes or types. While GAS is better for player systems, enemy systems don't share the same benefit, which is why I ultimately decided to use only Type Object.

What pattern was the *most critical* to your design?

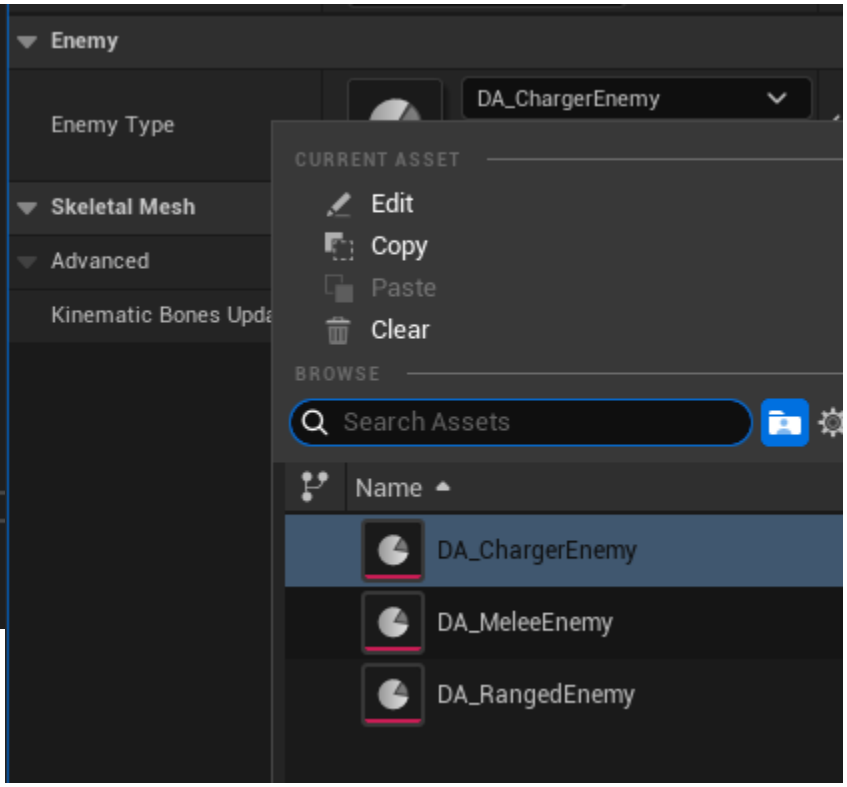
Obviously, Type Object is the main Pattern I am showcasing, as it is needed to showcase how simple it is to create new and different enemies based on the stats I have defined. What is critical about it is how easy it is to add new elements that all new enemy types have access to if they need it (like a projectile to shoot; maybe one enemy can shoot 3 in a cone). If I were to do enemy behavior logic differently, instead of doing an enum-based branching, I would use a behavior tree (next assignment).

```
if (Distance <= EnemyType->AttackRange)
{
    switch (EnemyType->AttackType)
    {
    case EEnemyAttackType::Melee:
        HandleMeleeAttack();
        break;

    case EEnemyAttackType::Ranged:
        HandleRangedAttack();
        break;

    case EEnemyAttackType::Charger:
        HandleChargeAttack();
        break;

    }
}
```



What valuable information did you learn for yourself while doing this project?

I learned the difference between subclass-driven design and data-driven design. Implementing the Type Object pattern showed me how powerful it is to separate behavior definitions from runtime logic, allowing systems to scale without duplicating code. Once the Type Object system was in place, extending enemy types became surprisingly simple. Creating a new enemy only

required making a new Data Asset and adjusting values rather than writing new classes or modifying existing logic.

Catch All

One interesting implementation detail was transitioning from timer-based projectile spawning to animation-synchronized spawning. Although it required iteration and debugging, it deepened my understanding of animation-driven gameplay systems. This is something that GAS was designed for.

Time-Spent

I spent 12-16 hours implementing the 2 systems, but then cutting the SandClass sandbox. Type Object took me the longest to figure out some quirks, like how the data assets don't appear in the editor if you don't compile at source. Filming the video took many attempts to get it right and condensed into 4 minutes.